

MAT 3510 — Homework 1 Study Notes

Sample Variance & Polynomial Addition

These are notes on the *math* behind the two problems, plus MATLAB mechanics that trip people up. They are meant to help you write the functions yourself, not to be handed in.

Problem 1: Sample Variance

The formula

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

What each piece means

- $\bar{x}$ is the sample mean: the average of your n data points.
- $x_i - \bar{x}$ is the *deviation* of point i from the average. Some deviations are positive, some negative, and they sum to zero by definition of the mean.
- Squaring, $(x_i - \bar{x})^2$, makes every term non-negative (so positive and negative deviations don't cancel) and penalizes large deviations more heavily than small ones.
- Summing the squared deviations gives the total spread in the data.
- Dividing by $n - 1$ (rather than n) rescales that total into an average-ish quantity.

Why $n - 1$ and not n ? (Bessel's correction)

If you knew the true population mean μ , then $\frac{1}{n} \sum (x_i - \mu)^2$ would be the natural, unbiased estimate of the population variance. But we don't know μ — we only have $\bar{x}$, which is computed *from the same sample*. It is a fact that

$$\bar{x} = \arg \min_c \sum_{i=1}^n (x_i - c)^2,$$

i.e. $\bar{x}$ is exactly the value of c that minimizes the sum of squared deviations. So $\sum (x_i - \bar{x})^2 \leq \sum (x_i - \mu)^2$ always, which means using $\bar{x}$ in place of μ systematically *understates* the true spread. Dividing by $n - 1$ instead of n exactly corrects this bias on average. This is also why MATLAB's built-in `var` divides by $n - 1$ by default — so your `samplevar` should agree with it.

MATLAB mechanics: .^2 vs 2

This is the most common bug on this problem. In MATLAB, $\wedge$ is *matrix power* — it only makes sense for square matrices (or scalars). If $\mathbf{x}$ is a vector and you write $\mathbf{x}^2$, MATLAB will error (or, for a scalar, just square it, which is why the bug is easy to miss when testing on a single number).

What you actually want is *elementwise* squaring: put a dot before the operator, $\mathbf{x}.\wedge 2$. The dot family ($\text{.}\wedge$, $\text{.}\ast$, $\text{.}/$) means “do this operation entry-by-entry,” as opposed to the matrix versions of those operations.

```
x = [2 4 4 4 5 5 7 9];  
x.^2 % correct: squares each entry -> [4 16 16 16 25 25 49 81]  
x^2 % ERROR for a non-square, non-scalar x
```

Recipe for `samplevar(x)`

1. `n = length(x)`
2. `xbar = mean(x)`
3. `deviations = x - xbar` (MATLAB broadcasts the scalar `xbar` against every entry of `x` automatically)
4. `squared = deviations.^2`
5. `s2 = sum(squared) / (n - 1)`

Check your work: `samplevar(x)` should equal MATLAB’s built-in `var(x)` on the same vector, for any $\mathbf{x}$.

Problem 2: Adding Polynomials

How a polynomial becomes a vector

A polynomial like $3x^2 + 5x - 2$ is represented by its coefficients, listed from the *highest* degree down to the constant term:

$$3x^2 + 5x - 2 \longleftrightarrow [3, 5, -2].$$

The key subtlety: the degree that a coefficient represents depends on *where it sits relative to the end of the vector*, not on a fixed index. In a length-3 vector, position 1 is the x^2 coefficient; in a length-4 vector, position 1 is the x^3 coefficient. The *last* entry is always the constant term (x^0), regardless of length.

Why you can’t just add the vectors directly

Adding two polynomials means adding the coefficients of matching degrees:

$$(a_2x^2 + a_1x + a_0) + (b_3x^3 + b_2x^2 + b_1x + b_0) = b_3x^3 + (a_2 + b_2)x^2 + (a_1 + b_1)x + (a_0 + b_0).$$

If $p = [a_2, a_1, a_0]$ has length 3 and $q = [b_3, b_2, b_1, b_0]$ has length 4, then $\mathbf{p} + \mathbf{q}$ in MATLAB either errors (lengths differ) or, if you tried to force it, would misalign degree-2 terms in p with degree-3 terms in q . You must first align them *by degree*.

The trick: pad with leading zeros

Treat the shorter polynomial as though it has zero coefficients for the missing high-degree terms. Concretely, $x^2 + 2x + 3$ is really $0 \cdot x^3 + 1 \cdot x^2 + 2 \cdot x + 3$, so as a length-4 vector it is $[0, 1, 2, 3]$. Once both vectors are padded on the **left** (the high-degree end) to the same length, position k in one vector lines up with the same degree as position k in the other, and ordinary elementwise addition is correct.

$$\begin{array}{rrrr} p = x^2 + 2x + 3 & 1 & 2 & 3 \\ \text{padded to length 4} & 0 & 1 & 2 & 3 \\ q = 5x^3 + x + 2 & 5 & 0 & 1 & 2 \\ \hline p + q & 5 & 1 & 3 & 5 \end{array}$$

so $p + q = 5x^3 + x^2 + 3x + 5$.

Common mistake: padding on the right instead of the left. That would shift every existing coefficient down by one degree instead of inserting zero coefficients for the missing high-degree terms — the result would be a completely different (wrong) polynomial.

Recipe for polyadd(p, q)

1. `np = length(p), nq = length(q)`
2. `n = max(np, nq)` (the target length — the higher-degree input doesn't need padding)
3. Pad the shorter one on the left: `pp = [zeros(1, n-np), p]`, `qq = [zeros(1, n-nq), q]`
4. `r = pp + qq`

Check your work by hand on an example with p and q of different lengths, e.g. $p = [1, 2, 3]$, $q = [5, 0, 1, 2]$ should give $[5, 1, 3, 5]$.

General MATLAB tips for this homework

- **File name = function name.** For MATLAB to recognize `samplevar` as a callable function, it must live in a file named exactly `samplevar.m` (same for `polyadd.m`). A file like `hw1.m` containing `function s2 = samplevar(x)` will confuse MATLAB about what the callable name is.
- **Test against a known answer.** For Problem 1, compare against the built-in `var`. For Problem 2, pick small examples you can add by hand.
- **Vector vs. scalar operators.** Whenever you're doing an operation "to every entry of a list" (square, multiply, divide), reach for the dot operators `.^`, `.*`, `./` rather than their plain counterparts, unless you specifically mean linear algebra (matrix multiplication, matrix powers).